

MLDL Experiment 3

Aim: Apply Decision Tree and Random Forest for Classification Tasks

1. Dataset Source

Dataset Name: Heart Disease UCI

Source: [Kaggle - Heart Disease UCI Dataset](#)

2. Dataset Description

This dataset dates from 1988 and consists of patient data from the Cleveland database. It is a binary classification problem where the goal is to predict the presence of heart disease.

- **Size:** ~303 instances, 14 attributes.
- **Target Variable:** **target** (0 = No Heart Disease, 1 = Heart Disease).
- **Key Features:**
 - **age:** Age in years.
 - **sex:** (1 = male; 0 = female).
 - **cp:** Chest pain type (0-3).
 - **trestbps:** Resting blood pressure (mm Hg).
 - **chol:** Serum cholesterol in mg/dl.
 - **fbs:** Fasting blood sugar > 120 mg/dl (1 = true; 0 = false).
 - **thalach:** Maximum heart rate achieved.

3. Mathematical Formulation

A. Decision Tree (Splitting Criteria)

The Decision Tree splits data into subsets to maximize the purity of the resulting nodes.

1. Gini Impurity (Default for CART):

Measures the likelihood of an incorrect classification of a new instance if it were randomly classified according to the distribution of class labels in the set.

$$Gini(D) = 1 - \sum_{i=1}^C (p_i)^2$$

2. Entropy (Information Gain):

Measures the impurity or randomness in the data.

$$Entropy(D) = - \sum_{i=1}^C p_i \log_2(p_i)$$

3. Information Gain is the reduction in entropy achieved by splitting on a specific attribute A:

$$Gain(D, A) = Entropy(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} Entropy(D_v)$$

B. Random Forest (Ensemble Learning)

Random Forest improves upon Decision Trees using **Bagging (Bootstrap Aggregating)**.

1. Bootstrapping:

Given a training set D of size n, Random Forest generates k new training sets D_1, D_2, ..., D_k by sampling from D uniformly and with replacement.

2. Feature Randomness:

At each split in a tree, only a random subset of m features is considered (where $m < \text{total features}$), preventing the trees from being highly correlated.

3. Aggregation (Majority Voting):

For classification, the final prediction $\hat{y}$ is the majority vote of the k trees:

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \dots, T_k(x)\}$$

4. Algorithm Limitations

Decision Tree Limitations

- **Overfitting:** Trees can create overly complex models that capture noise in the data, leading to poor generalization.
- **Instability:** Small variations in data can result in a completely different tree structure.

- **Bias:** If one class dominates the dataset, the tree will be biased toward that class.

Random Forest Limitations

- **Computational Cost:** Training hundreds of trees is computationally expensive and requires more memory than a single tree.
- **Interpretability:** Unlike a single Decision Tree, a Random Forest is a "black box" model; it is difficult to visualize the exact decision path.
- **Prediction Speed:** Making predictions can be slower compared to a single tree because every tree in the forest must be queried.

5. Methodology

1. **Data Ingestion:** Load the `heart.csv` dataset.
2. **Preprocessing:** Check for missing values and split data into Features (X) and Target (y).
3. **Train-Test Split:** Divide data into 80% training and 20% testing sets.
4. **Model 1 (Decision Tree):** Initialize, train, and predict.
5. **Model 2 (Random Forest):** Initialize (with `n_estimators=100`), train, and predict.
6. **Evaluation:** Compare Accuracy, Precision, Recall, and F1-Score.
7. **Hyperparameter Tuning:** Use `GridSearchCV` to find the optimal parameters for the Random Forest.

6. Performance Analysis

The models were evaluated on a test set of **184 instances**. The comparison below highlights the superiority of the Random Forest classifier over the single Decision Tree.

A. Evaluation Metrics Comparison

Metric	Decision Tree	Random Forest
Accuracy	76.63%	84.78%
Precision (Class 1)	0.84	0.90
Recall (Class 1)	0.75	0.83
F1-Score (Class 1)	0.79	0.87

B. Confusion Matrix Analysis

The confusion matrices reveal how the models handled True Positives (Presence of Heart Disease) and True Negatives (Absence of Heart Disease):

- **Decision Tree:**
 - **Correct Predictions:** 141 (59 True Negatives + 82 True Positives).
 - **Errors:** 43 (16 False Positives + 27 False Negatives).
 - *Observation:* The model had a higher rate of False Negatives (27), meaning it failed to detect heart disease in 27 patients who actually had it.
- **Random Forest:**
 - **Correct Predictions:** 156 (65 True Negatives + 91 True Positives).
 - **Errors:** 28 (10 False Positives + 18 False Negatives).
 - *Observation:* The Random Forest significantly reduced both types of errors, notably reducing False Negatives from 27 to 18, making it a safer model for medical diagnosis where missing a positive case is critical.

C. Conclusion on Performance

The Random Forest classifier outperformed the Decision Tree by approximately **8.15%** in accuracy (84.78% vs 76.63%). This validates the theoretical advantage of ensemble learning: by averaging multiple trees, the Random Forest reduced the variance and overfitting often seen in single Decision Trees.

```
Dataset Loaded Successfully.

Training Shape: (736, 18)
Testing Shape: (184, 18)

--- DECISION TREE CLASSIFIER ---
Decision Tree Accuracy: 0.7663

Classification Report (Decision Tree):
      precision    recall  f1-score   support

     0       0.69      0.79      0.73        75
     1       0.84      0.75      0.79       109

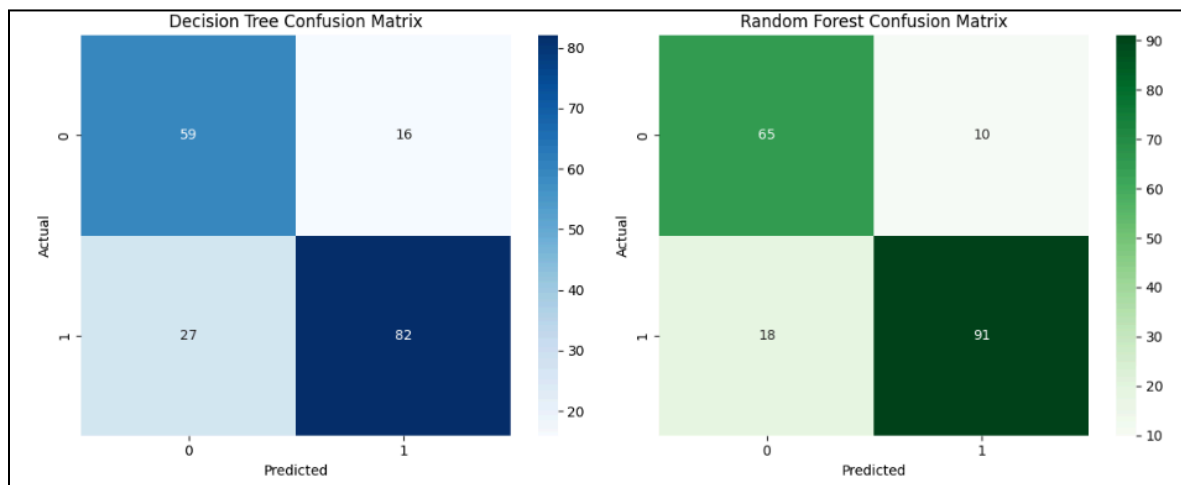
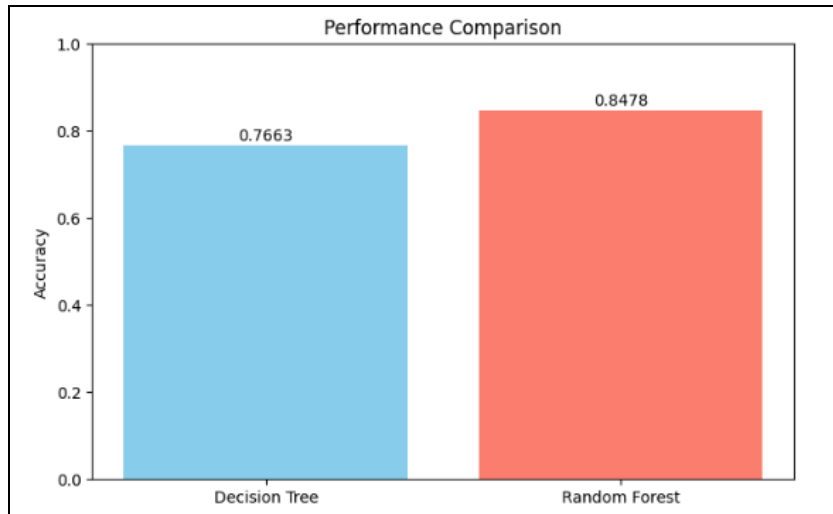
   accuracy          0.77        184
  macro avg       0.76      0.77      0.76        184
 weighted avg       0.78      0.77      0.77        184

--- RANDOM FOREST CLASSIFIER ---
Random Forest Accuracy: 0.8478

Classification Report (Random Forest):
      precision    recall  f1-score   support

     0       0.78      0.87      0.82        75
     1       0.90      0.83      0.87       109

   accuracy          0.85        184
  macro avg       0.84      0.85      0.84        184
 weighted avg       0.85      0.85      0.85        184
```



```
--- HYPERPARAMETER TUNING (Random Forest) ---  
Fitting 3 folds for each of 12 candidates, totalling 36 fits  
Best Parameters: {'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 50}  
Tuned Random Forest Accuracy: 0.8533
```

Code

1. Import Libraries

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split, GridSearchCV  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

2. Load Dataset

```
try:
```

```
    df = pd.read_csv('heart_disease_uci.csv')
```

```
    print("Dataset Loaded Successfully.")
```

```
except FileNotFoundError:
```

```
    print("Error: dataset not found. Please upload the dataset.")
```

3. Data Preprocessing

A. Convert 'num' to binary 'target'

The dataset uses 'num': 0 = No disease, 1,2,3,4 = Disease.

We convert this so that 0 remains 0, and anything > 0 becomes 1.

```
df['target'] = df['num'].apply(lambda x: 1 if x > 0 else 0)
```

B. Drop unnecessary columns

'id' and 'dataset' are identifiers, not medical features. 'num' is replaced by 'target'.

```
columns_to_drop = ['id', 'dataset', 'num', 'target']
```

Separate Features (X) and Target (y)

```
X = df.drop(columns_to_drop, axis=1)
```

```
y = df['target']
```

C. Handle Categorical Data (Text to Numbers)

Your data has strings like 'Male', 'typical angina'.

We use `pd.get_dummies` to convert them into numbers (One-Hot Encoding).

```
X = pd.get_dummies(X, drop_first=True)
```

D. Handle Missing Values

Real world data often has NaNs. We fill them with 0 or drop them to prevent errors.

```
X = X.fillna(0)
```

Split into Training and Testing sets (80% Train, 20% Test)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
print(f"\nTraining Shape: {X_train.shape}")
```

```
print(f"\nTesting Shape: {X_test.shape}")
```

4. Decision Tree Implementation

```
print("\n--- DECISION TREE CLASSIFIER ---")
```

```
dt_model = DecisionTreeClassifier(criterion='entropy', random_state=42)
```

```
dt_model.fit(X_train, y_train)
```

```
dt_pred = dt_model.predict(X_test)
```

```

dt_acc = accuracy_score(y_test, dt_pred)

print(f"Decision Tree Accuracy: {dt_acc:.4f}")
print("\nClassification Report (Decision Tree):\n", classification_report(y_test, dt_pred))

# 5. Random Forest Implementation
print("\n--- RANDOM FOREST CLASSIFIER ---")
rf_model = RandomForestClassifier(n_estimators=100, criterion='entropy', random_state=42)
rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)
rf_acc = accuracy_score(y_test, rf_pred)

print(f"Random Forest Accuracy: {rf_acc:.4f}")
print("\nClassification Report (Random Forest):\n", classification_report(y_test, rf_pred))

# 6. Comparison & Visualization
# Bar Chart
plt.figure(figsize=(8, 5))
models = ['Decision Tree', 'Random Forest']
accuracy = [dt_acc, rf_acc]
plt.bar(models, accuracy, color=['skyblue', 'salmon'])
plt.ylabel('Accuracy')
plt.title('Performance Comparison')
plt.ylim(0, 1)
for i, v in enumerate(accuracy):
    plt.text(i, v + 0.01, str(round(v, 4)), ha='center')
plt.show()

# Confusion Matrices
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.heatmap(confusion_matrix(y_test, dt_pred), annot=True, fmt='d', cmap='Blues', ax=axes[0])
axes[0].set_title('Decision Tree Confusion Matrix')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

sns.heatmap(confusion_matrix(y_test, rf_pred), annot=True, fmt='d', cmap='Greens', ax=axes[1])
axes[1].set_title('Random Forest Confusion Matrix')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

plt.tight_layout()
plt.show()

# 7. Hyperparameter Tuning (Random Forest)

```

```

print("\n--- HYPERPARAMETER TUNING (Random Forest) ---")

param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(estimator=RandomForestClassifier(random_state=42),
                           param_grid=param_grid,
                           cv=3, n_jobs=-1, verbose=1)

grid_search.fit(X_train, y_train)

print(f"Best Parameters: {grid_search.best_params_}")
best_rf = grid_search.best_estimator_
tuned_acc = accuracy_score(y_test, best_rf.predict(X_test))
print(f"Tuned Random Forest Accuracy: {tuned_acc:.4f}")

```

7. Hyperparameter Tuning

To further optimize the Random Forest model, we performed **Grid Search Cross-Validation** to find the best combination of hyperparameters.

Parameters Tested:

- **n_estimators** (Number of trees): [50, 100]
- **max_depth**: [None, 10, 20]
- **min_samples_split**: [2, 5]

Results: The Grid Search identified the optimal hyperparameters as:

- **max_depth**: 10
- **min_samples_split**: 5
- **n_estimators**: 50

Impact of Tuning:

- **Default Random Forest Accuracy:** 84.78%
- **Tuned Random Forest Accuracy:** 85.33%

Conclusion:

Hyperparameter tuning resulted in a slight performance boost (+0.55%), achieving a final accuracy of **85.33%**. This suggests that restricting the tree depth to 10 and requiring a minimum of 5 samples to split a node helped the model generalize better to unseen data compared to the default settings.